

Architecture technique — GarageFlow

1. Objectif de l'architecture

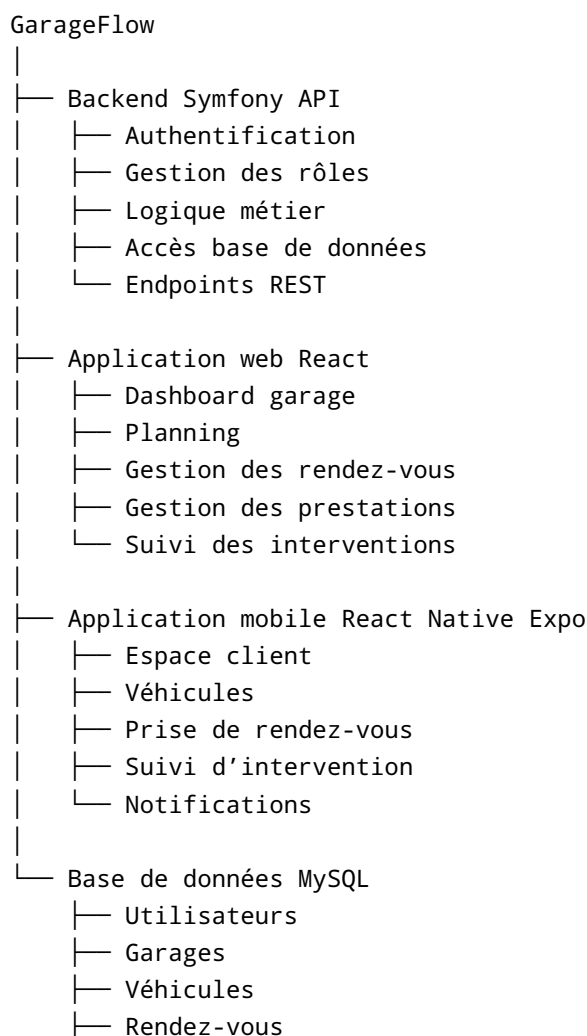
L'objectif de l'architecture technique de GarageFlow est de construire une application claire, maintenable, sécurisée et évolutive.

Le projet est composé de trois parties principales :

- un backend Symfony exposant une API REST sécurisée ;
- une application web React destinée aux garages ;
- une application mobile React Native Expo destinée aux clients.

Cette séparation permet d'avoir une seule API centrale consommée par plusieurs interfaces.

2. Vue globale de l'architecture



3. Stack technique retenue

Partie	Technologie
Backend	Symfony API
ORM	Doctrine ORM
Base de données	MySQL
Migrations	Doctrine Migrations
Authentification	JWT
Application web garage	React + Vite
Application mobile client	React Native + Expo
Style	Tailwind CSS côté web
Communication	API REST
Versioning	Git / GitHub
Documentation	Markdown dans le dossier docs

4. Justification des choix techniques

Symfony API

Symfony est utilisé pour créer le backend de l'application.

Ce choix permet de structurer le projet avec :

- des contrôleurs ;
- des services métier ;
- des entités Doctrine ;
- des repositories ;
- des validateurs ;
- une gestion de la sécurité ;
- une architecture claire et maintenable.

Symfony est adapté au projet car GarageFlow repose sur beaucoup de logique métier : réservation, gestion des créneaux, rôles, isolation des données par garage, interventions, statuts et notifications.

Doctrine ORM

Doctrine ORM permet de faire le lien entre les objets PHP et les tables SQL.

Les entités Symfony représentent les tables de la base de données.

Exemples :

- User → table utilisateur ;
- Garage → table garage ;
- Vehicle → table vehicule ;
- Appointment → table rendez_vous ;
- Intervention → table intervention.

Doctrine permet de centraliser l'accès aux données dans les repositories et d'éviter de disperser les requêtes SQL dans les contrôleurs.

MySQL

MySQL est choisi comme base de données relationnelle.

Ce choix est cohérent car les données du projet sont fortement liées :

- un garage possède plusieurs prestations ;
 - un client possède plusieurs véhicules ;
 - un rendez-vous relie un client, un véhicule, un garage et une prestation ;
 - une intervention dépend d'un rendez-vous confirmé ;
 - les statuts et notifications sont liés à l'intervention.
-

React + Vite

React est utilisé pour l'application web destinée aux garages.

Ce choix permet de construire une interface moderne avec des composants réutilisables.

L'application web doit permettre au garage de gérer :

- ses rendez-vous ;
- son planning ;
- ses prestations ;
- ses horaires ;
- ses interventions ;
- ses notifications ;
- ses notes internes.

Vite permet d'avoir un environnement de développement rapide et léger.

React Native Expo

React Native Expo est utilisé pour l'interface mobile client.

Ce choix permet de proposer une vraie expérience mobile plutôt qu'un simple site responsive.

Le client pourra :

- se connecter ;
- gérer ses véhicules ;
- prendre rendez-vous ;
- suivre l'avancement de son intervention ;
- consulter ses notifications.

Cette approche permet de répondre clairement au besoin d'une interface mobile Android et/ou iOS.

5. Organisation du projet

Le projet sera organisé sous forme de monorepo.

```
garageflow/
├── backend/
│   ├── src/
│   │   ├── Entity/
│   │   ├── Repository/
│   │   ├── Controller/
│   │   ├── Service/
│   │   ├── Security/
│   │   ├── DTO/
│   │   ├── Validator/
│   │   └── EventSubscriber/
│   ├── migrations/
│   ├── config/
│   ├── tests/
│   └── .env.example
├── web/
│   ├── src/
│   │   ├── pages/
│   │   ├── components/
│   │   ├── services/api/
│   │   ├── hooks/
│   │   ├── contexts/
│   │   └── routes/
└──
```

```
|   └─ .env.example
|
|   └─ mobile/
|       └─ src/
|           ├── screens/
|           ├── components/
|           ├── services/api/
|           ├── navigation/
|           └─ contexts/
|       └─ .env.example
|
|   └─ docs/
|       ├── fonctionnelle/
|       ├── technique/
|       ├── mcd/
|       ├── mld/
|       ├── api/
|       └─ oral/
```

6. Architecture backend

Principe général

Le backend suit une architecture en couches.

```
Controller
↓
DTO / Validator
↓
Service métier
↓
Repository
↓
Doctrine ORM
↓
Base de données MySQL
```

Chaque couche a une responsabilité précise.

Controller

Les contrôleurs reçoivent les requêtes HTTP.

Ils ne doivent pas contenir toute la logique métier.

Leur rôle est de :

- recevoir la requête ;
- vérifier l'utilisateur connecté ;
- appeler le bon service ;
- retourner une réponse JSON.

Exemple :

AppointmentController

Rôle :

- créer une demande de rendez-vous ;
- accepter un rendez-vous ;
- annuler un rendez-vous ;
- lister les rendez-vous d'un garage ou d'un client.

DTO

Les DTO permettent de structurer les données reçues depuis le frontend ou le mobile.

Exemple :

CreateAppointmentDTO

Champs possibles :

- garageId ;
- vehicleId ;
- serviceId ;
- dateDebut ;
- commentaireClient.

Le DTO permet d'éviter de manipuler directement les entités avec les données envoyées par l'utilisateur.

Validator

Les validateurs permettent de vérifier les données avant traitement.

Exemples de validations :

- le véhicule appartient au client connecté ;

- le garage est actif ;
 - la prestation appartient au garage ;
 - la date du rendez-vous est dans le futur ;
 - le créneau est disponible ;
 - le rendez-vous ne chevauche pas un autre rendez-vous.
-

Service métier

Les services contiennent la logique métier principale.

Exemples :

```
AppointmentService  
GarageService  
InterventionService  
NotificationService  
AvailabilityService  
LogActionService
```

Le service évite de mettre la logique métier directement dans les contrôleurs.

Exemple :

```
AppointmentService
```

Responsabilités :

- vérifier la disponibilité d'un créneau ;
 - créer une demande de rendez-vous ;
 - accepter ou refuser un rendez-vous ;
 - créer une intervention si le rendez-vous est confirmé ;
 - générer une notification ;
 - journaliser l'action.
-

Repository

Les repositories centralisent les requêtes vers la base de données.

Exemples :

```
AppointmentRepository  
VehicleRepository
```

```
GarageRepository
InterventionRepository
```

Exemples de méthodes :

```
findAppointmentsByGarageAndDate()
findClientAppointments()
hasAppointmentConflict()
findVehicleHistory()
```

Entity

Les entités représentent les tables de la base de données.

Exemples :

```
User
Role
Garage
Vehicle
Appointment
Intervention
Notification
ActionLog
```

Elles sont gérées par Doctrine ORM.

7. Communication entre les applications

Les deux interfaces consomment la même API REST.

```
React Web Dashboard
    ↓
API REST Symfony
    ↑
React Native Mobile
```

L'application web et l'application mobile ne communiquent pas directement avec la base de données.

Toutes les opérations passent par le backend afin de garantir :

- la sécurité ;
- la validation des données ;

- l'application des règles métier ;
- la traçabilité.

8. API REST prévue

Authentification

Méthode	Endpoint	Rôle
POST	/api/auth/register	Créer un compte client
POST	/api/auth/login	Se connecter
GET	/api/auth/me	Récupérer l'utilisateur connecté

Garages

Méthode	Endpoint	Rôle
GET	/api/garages	Lister les garages actifs
GET	/api/garages/{id}	Voir une fiche garage
POST	/api/admin/garages	Créer un garage
PATCH	/api/garages/{id}	Modifier son garage

Prestations

Méthode	Endpoint	Rôle
GET	/api/garages/{id}/services	Lister les prestations d'un garage
POST	/api/garage/services	Créer une prestation
PATCH	/api/garage/services/{id}	Modifier une prestation
DELETE	/api/garage/services/{id}	Désactiver une prestation

Véhicules

Méthode	Endpoint	Rôle
GET	/api/client/vehicles	Lister ses véhicules
POST	/api/client/vehicles	Ajouter un véhicule

Méthode	Endpoint	Rôle
PATCH	/api/client/vehicles/{id}	Modifier un véhicule
DELETE	/api/client/vehicles/{id}	Supprimer ou désactiver un véhicule

Rendez-vous

Méthode	Endpoint	Rôle
POST	/api/client/appointments	Créer une demande de rendez-vous
GET	/api/client/appointments	Voir ses rendez-vous
GET	/api/garage/appointments	Voir les rendez-vous du garage
PATCH	/api/garage/appointments/{id}/accept	Accepter un rendez-vous
PATCH	/api/garage/appointments/{id}/refuse	Refuser un rendez-vous
PATCH	/api/appointments/{id}/cancel	Annuler un rendez-vous

Interventions

Méthode	Endpoint	Rôle
GET	/api/client/interventions	Voir ses interventions
GET	/api/garage/interventions	Voir les interventions du garage
GET	/api/garage/interventions/{id}	Voir le détail d'une intervention
PATCH	/api/garage/interventions/{id}/status	Changer le statut
POST	/api/garage/interventions/{id}/notes	Ajouter une note interne

Notifications

Méthode	Endpoint	Rôle
GET	/api/notifications	Voir ses notifications
PATCH	/api/notifications/{id}/read	Marquer une notification comme lue

9. Sécurité

Authentification JWT

L'authentification sera basée sur un token JWT.

Après connexion, l'utilisateur reçoit un token.

Ce token est envoyé dans les requêtes API suivantes.

```
Authorization: Bearer <token>
```

Le backend vérifie le token avant de donner accès aux routes protégées.

Gestion des rôles

Les rôles prévus sont :

- ROLE_ADMIN ;
- ROLE_GERANT ;
- ROLE_EMPLOYE ;
- ROLE_CLIENT.

Chaque route sensible sera protégée selon le rôle attendu.

Exemples :

- un client peut créer un rendez-vous ;
 - un employé peut gérer les rendez-vous de son garage ;
 - un gérant peut gérer les prestations et les horaires ;
 - un admin peut gérer les garages.
-

Isolation par garage

L'isolation par garage est une règle centrale.

Un employé ou un gérant ne doit voir que les données de son garage.

Exemple :

Un gérant du Garage A ne doit jamais pouvoir modifier une prestation du Garage B.

Cette règle doit être appliquée côté backend, pas seulement côté frontend.

Validation backend

Le backend reste la source de vérité.

Même si le frontend effectue des validations, le backend doit toujours vérifier les données.

Exemples :

- email valide ;
 - mot de passe hashé ;
 - créneau disponible ;
 - véhicule appartenant au client connecté ;
 - prestation appartenant au garage ;
 - intervention liée au bon garage ;
 - note interne non visible côté client.
-

Journalisation

Les actions importantes sont enregistrées dans la table LOG_ACTION.

Exemples :

- création d'un rendez-vous ;
- acceptation d'un rendez-vous ;
- refus d'un rendez-vous ;
- annulation d'un rendez-vous ;
- changement de statut d'intervention ;
- création ou modification d'une prestation.

La journalisation améliore la traçabilité et la maintenance.

10. Architecture frontend web

L'application web est destinée aux garages.

Elle doit être optimisée pour une utilisation desktop et tablette.

Structure prévue

```
web/src/  
|  
├─ pages/  
|   └─ LoginPage.tsx  
|   └─ DashboardPage.tsx  
|   └─ AppointmentsPage.tsx
```

```
|   |   | PlanningPage.tsx
|   |   | ServicesPage.tsx
|   |   | InterventionDetailPage.tsx
|   |   | SettingsPage.tsx
|   |
|   | components/
|   |   | Layout/
|   |   | Sidebar/
|   |   | Button/
|   |   | Card/
|   |   | Table/
|   |   | Form/
|   |   | StatusBadge/
|   |
|   | services/api/
|   |   | apiClient.ts
|   |   | authApi.ts
|   |   | appointmentApi.ts
|   |   | garageApi.ts
|   |   | interventionApi.ts
|   |
|   | hooks/
|   |   | useAuth.ts
|   |   | useAppointments.ts
|   |   | useInterventions.ts
|   |
|   | contexts/
|   |   | AuthContext.tsx
|   |
|   | routes/
|   |   | AppRouter.tsx
```

Pages principales web

L'application web garage contiendra :

- page de connexion ;
 - dashboard ;
 - planning ;
 - liste des rendez-vous ;
 - fiche rendez-vous ;
 - gestion des prestations ;
 - gestion des horaires ;
 - détail intervention ;
 - notifications.
-

11. Architecture mobile

L'application mobile est destinée aux clients.

Elle doit être pensée pour un parcours simple et rapide.

Structure prévue

```
mobile/src/  
|  
├── screens/  
│   ├── LoginScreen.tsx  
│   ├── RegisterScreen.tsx  
│   ├── HomeScreen.tsx  
│   ├── GarageDetailScreen.tsx  
│   ├── ServiceSelectionScreen.tsx  
│   ├── SlotSelectionScreen.tsx  
│   ├── VehicleSelectionScreen.tsx  
│   ├── AppointmentConfirmationScreen.tsx  
│   ├── MyAppointmentsScreen.tsx  
│   ├── InterventionTrackingScreen.tsx  
│   └── NotificationsScreen.tsx  
├── components/  
│   ├── Button/  
│   ├── Card/  
│   ├── Input/  
│   ├── StatusTimeline/  
│   └── LoadingState/  
├── services/api/  
│   ├── apiClient.ts  
│   ├── authApi.ts  
│   ├── vehicleApi.ts  
│   ├── appointmentApi.ts  
│   └── notificationApi.ts  
├── navigation/  
│   ├── AuthNavigator.tsx  
│   ├── ClientNavigator.tsx  
│   └── RootNavigator.tsx  
└── contexts/  
    └── AuthContext.tsx
```

Écrans principaux mobile

L'application mobile client contiendra :

- inscription ;
- connexion ;
- accueil ;
- page garage ;
- choix prestation ;
- choix créneau ;
- choix véhicule ;
- confirmation rendez-vous ;
- liste des rendez-vous ;
- suivi intervention ;
- notifications.

12. Gestion des erreurs et feedbacks UX

L'application doit afficher des retours clairs à l'utilisateur.

Exemples :

- message de succès après création d'un rendez-vous ;
- message d'erreur si le créneau n'est plus disponible ;
- loader pendant le chargement ;
- badge de statut sur les rendez-vous ;
- confirmation avant annulation ;
- message clair si aucun rendez-vous n'est trouvé ;
- notification visuelle après changement de statut.

Ces retours améliorent l'expérience utilisateur et réduisent les incompréhensions.

13. Performance

Plusieurs optimisations sont prévues.

Backend

- requêtes filtrées par garage ou utilisateur ;
- index sur les champs importants ;
- pagination sur les listes longues ;
- repositories dédiés aux requêtes complexes ;
- limitation des données renvoyées selon le rôle.

Frontend web

- composants réutilisables ;
- chargement uniquement des données nécessaires ;
- séparation des pages ;
- feedbacks de chargement ;
- filtres sur les rendez-vous.

Mobile

- écrans simples ;
 - peu de données chargées au démarrage ;
 - appels API ciblés ;
 - mise en cache légère possible pour l'utilisateur connecté.
-

14. Tests prévus

Tests fonctionnels

Exemples :

- création d'un compte client ;
- connexion ;
- ajout d'un véhicule ;
- création d'une demande de rendez-vous ;
- acceptation d'un rendez-vous ;
- création d'une intervention ;
- changement de statut ;
- génération d'une notification.

Tests de sécurité

Exemples :

- un client ne peut pas voir les véhicules d'un autre client ;
- un employé ne peut pas voir les rendez-vous d'un autre garage ;
- un client ne peut pas accéder aux notes internes ;
- une route gérant n'est pas accessible à un client.

Tests de validation

Exemples :

- impossible de réserver dans le passé ;
- impossible de réserver un créneau indisponible ;
- impossible d'ajouter une prestation sans durée ;
- impossible de créer deux fois le même véhicule pour un client.

15. Organisation Git

Le projet sera versionné avec GitHub.

Branches prévues

```
main
develop
feature/auth
feature/garage-dashboard
feature/appointments
feature/interventions
feature/mobile-client
feature/notifications
```

Convention de commits

Exemples :

```
feat(auth): add JWT login endpoint
feat(appointments): create appointment request
fix(security): restrict garage appointments access
docs(mcd): add data model documentation
```

Organisation GitHub

Le projet utilisera :

- issues GitHub ;
- branches par fonctionnalité ;
- pull requests si travail en équipe ;
- commits réguliers ;
- README d'installation ;
- documentation dans le dossier docs.

16. Déploiement prévu

Le déploiement n'est pas la priorité du MVP, mais l'architecture doit rester déployable.

Environnements possibles

- backend Symfony déployé sur un serveur ou une plateforme cloud ;
- base MySQL hébergée ;

- application web React buildée et déployée ;
- application mobile testable via Expo Go.

Variables d'environnement

Chaque partie du projet aura un fichier `.env.example`.

Exemples :

```
DATABASE_URL=  
JWT_SECRET=  
API_BASE_URL=  
MAILER_DSN=
```

Les fichiers `.env` réels ne seront pas commités sur GitHub.

17. Synthèse orale

L'architecture de GarageFlow repose sur une séparation claire entre le backend, l'application web et l'application mobile.

Le backend Symfony expose une API REST sécurisée. Il contient la logique métier, l'authentification, les rôles, les validations, les services et l'accès à la base de données via Doctrine ORM.

L'application web React est destinée aux garages et permet de gérer les rendez-vous, le planning, les prestations et les interventions.

L'application mobile React Native Expo est destinée aux clients et permet de réserver un rendez-vous, gérer ses véhicules et suivre l'avancement d'une intervention.

Cette architecture permet de centraliser les règles métier dans le backend tout en proposant deux interfaces adaptées à deux usages différents : le garage sur web et le client sur mobile.